

InternVLA-N1: từ dữ liệu RGB-D đến hành động của robot

Dual-system đầy đủ: System 2 (VLM) → latent plan → System 1 (NavDP) — đã chạy end-to-end open-loop

● **Đã chạy thật**

NavDP standalone trên sample InternData-N1

● **Đã đối chiếu code**

Pipeline dual-system, latent plan, post-processing

● **Đã chạy open-loop**

Dual-system 7B trên Kaggle T4×2 (23/07); còn lại closed-loop

Hai luồng song song, hội tụ ở dual-system

A

Người A · Model & Local

- WSL2 + env internnav-s1, submodule, checkpoint NavDP
- NavDP standalone: RGB-D backbone, diffusion, critic
- Đọc internvla_n1_agent.py: cơ chế ghép S2 → latent → S1
- Chạy real-data trên laptop GPU + tài liệu I/O System 1

B

Người B · Cloud & Eval

- Kaggle T4×2: đóng gói checkpoint, chạy S2 (Qwen2.5-VL 7B)
- Dual-system 7B open-loop 23/07: GATE 1/2/3, metric S2, look-down 2 nhịp
- Prompt/parser S2 · data contract LeRobot · theo dõi yêu cầu server

Hội tụ (Ngày 4–5): ghép dual-system, chạy thử, sync nhóm SIM

Bước kế tiếp chung: closed-loop SR/SPL trên Habitat/Isaac (server ≥24GB).

Năm phần

01

Bối cảnh & kiến trúc

Vì sao tách slow/fast; ba khái niệm dễ nhầm lẫn

02

Data contract InternData-N1

Cấu trúc LeRobot, preview thật, tiền xử lý RGB-D-pose

03

System 1 — NavDP standalone

I/O contract, RGB-D backbone, diffusion, critic

Phần 03 là trọng tâm

04

Điểm ghép S2 → latent → S1

Learned latent queries, token compressor, dual async

05

Bảng chứng, lỗi thường gặp & tiến độ

Kết quả real-data, failure modes, gate checkpoint 7B

Một lệnh, hai loại khó khăn khác nhau

“Đi qua cửa rồi rẽ trái tới ghế đỏ”

1 Hiểu & lập kế hoạch

- Hiểu ngôn ngữ tự nhiên dài
- Nhận diện đồ vật, giữ ngữ cảnh cả episode
- Chọn mục tiêu tiếp theo hợp lý

Chậm · không cần chạy mọi frame

2 Điều khiển thời gian thực

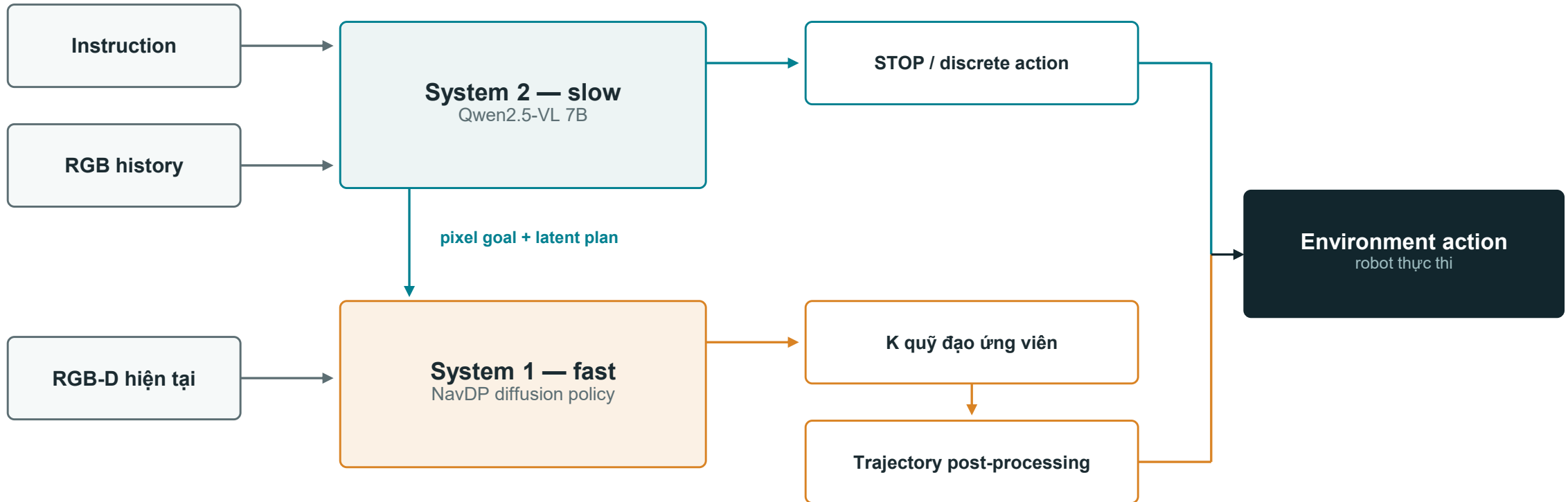
- Biến mục tiêu thành chuyển động ngắn hạn
- Tránh chướng ngại theo quan sát hiện tại
- Phản ứng với thay đổi cục bộ

Nhanh · phải chạy liên tục

Vì sao không dùng một model duy nhất cho mọi frame?

VLM 7B quá chậm và quá đắt để chạy ở tần số điều khiển. InternVLA-N1 tách: System 2 suy luận thưa và tạo latent plan; System 1 sinh quỹ đạo cục bộ nhiều lần trong khi tái sử dụng chính latent plan đó.

Dual-system: ground slow, move fast



Điểm cốt lõi: trong dual-system, S2 không chỉ trả về một câu chữ hay một tọa độ pixel. Nó sinh ra latent plan — biểu diễn học được của “đi đâu tiếp theo”. NavDP nhận latent đó làm goal condition, chứ không phải một điểm (x, y, theta) dạng số.

Ba chỗ rất dễ nhầm lẫn

1 S2 image history $\neq$ S1 RGB-D memory

S2

tối đa num_history = 8 ảnh lịch sử + ảnh hiện tại, resize 384×384

S1 standalone

memory_size = 8 RGB ở 224×224, nhưng chỉ 1 depth hiện tại

S1 dual async

memory_size = 2: cặp RGB-D lúc lập plan + cặp hiện tại

Số 8 của S2 không đồng nghĩa với số 2 của S1 dual async.

2 Pixel goal $\neq$ latent plan

Pixel goal

toạ độ quan sát được, lưu theo thứ tự [y, x] — dùng để log/visualize

Latent plan

tensor hidden-state, không mang ý nghĩa toạ độ trực tiếp

Trong code

latent điều kiện hoá diffusion; pixel goal giữ frame “lúc lập plan”

Về nhầm $[x,y] \leftrightarrow [y,x]$ là lỗi visualize phổ biến.

3 NavDP standalone $\neq$ NavDP nhúng trong InternVLA-N1

Hai đường dùng chung khái niệm quỹ đạo nhưng KHÔNG nhận cùng một biểu diễn goal → xem slide sau.

NavDP standalone và NavDP trong dual-system

Thuộc tính	NavDP standalone (đã chạy thật)	NavDP trong dual-system (đã chạy thật)
Goal condition	Numeric point goal (x, y, theta)	Latent plan sinh bởi System 2
Goal encoder	Linear(3 → 384)	MLP 3584 → 384, rồi token compressor
RGB-D contract	8 RGB history + 1 depth hiện tại	2 RGB + 2 depth (async path)
Prediction horizon	24 bước	32 bước (predict_size=32, đo thật: s1_traj (33,2))
Diffusion steps	10 timestep	20 timestep
Critic ranking	Có, trong API standalone đã gọi	generate_traj() trả toàn bộ sample; post-process lấy mean
Trạng thái local	Đã chạy trên RTX 4050 Laptop	Đã chạy open-loop trên Kaggle T4×2 (23/07) — 101/128 frame ra latent

Cùng tên “NavDP” nhưng khác goal representation — không thể thay thế lẫn nhau ở tầng API.

PHẦN 02

Data contract InternData-N1

Từ file trên đĩa đến tensor mà NavDP thực sự nhận

Cấu trúc một trajectory theo chuẩn LeRobot

```
<root>/<sub_dataset>/<scene>/<trajectory>/
├─ data/chunk-000/
│   └─ episode_000000.parquet
├─ meta/
│   ├── episodes_stats.jsonl
│   ├── episodes.jsonl
│   ├── info.json
│   └─ tasks.jsonl
└─ videos/chunk-000/
    ├── observation.images.rgb/ 0.jpg ...
    └─ observation.images.depth/ 0.png ...
```

“Có RGB và depth” là chưa đủ. Để tạo point goal và trajectory label đúng, bắt buộc phải có camera geometry / pose.

RGB	Ngữ nghĩa cảnh cho S2; visual context cho S1
Depth	Hình học gần robot: chúngng ngại, khoảng cách
Camera intrinsic	Chiều điểm 3D/camera thành pixel
Camera extrinsic	Pose camera → đổi world sang local trajectory
action (Parquet)	Ở vln_n1 là chuỗi transform 4×4, KHÔNG phải [v, w]
Task / instruction	Input ngôn ngữ của System 2
Point cloud	Chỉ dùng cho obstacle critic khi huấn luyện/đánh giá

Sample InternData-N1: ba nhánh vln_ce / vln_n1 / vln_pe, định dạng khác nhau

✕ **vln_ce**

Có RGB, nhưng thiếu depth và cột camera-pose

Không dùng được cho adapter S1

✓ **vln_n1**

122 RGB JPG · 122 depth PNG · intrinsic · extrinsic · action

Sample phù hợp — đã chạy thật

✕ **vln_pe**

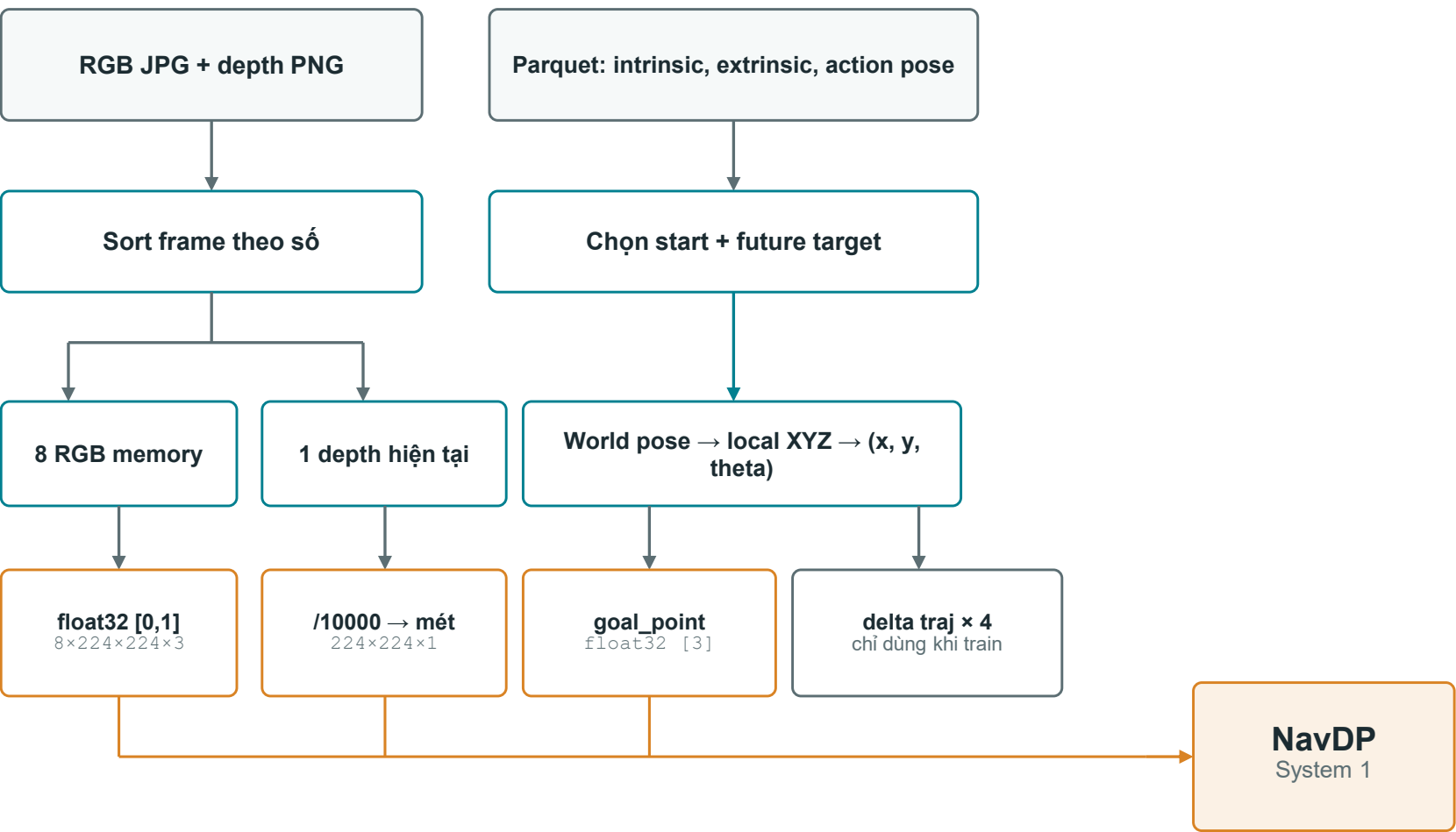
RGB/depth trong .npy, nhưng parquet không lộ camera matrices

Chưa dùng trực tiếp

Vì sao phải viết adapter riêng?

- Metadata preview dùng schema lồng dưới stats và thiếu meta/pointcloud.ply → không mở được bằng constructor directory-discovery của loader gốc.
- Adapter cung cấp compatibility view một episode: sắp tên frame theo số (2 trước 10), tái dùng nguyên các phép biến đổi image / depth / pose / point-goal / pixel-goal / action của upstream.
- Cấp obstacle set rỗng → kích hoạt neutral critic fallback sẵn có, không làm thay đổi tensor dùng cho inference.

Dataset được chuyển thành input NavDP như thế nào



Chọn frame & memory

```
start = (memory_size-1) * memory_digit  
..., start
```

memory_size	8
-------------	---

memory_digit	4 (khi không random)
--------------	----------------------

thiếu frame đầu	zero-pad
-----------------	----------

depth	chỉ ở start_step
-------	------------------

Không tạo lịch sử 8 depth frame

Lập depth thành 8 frame chính là nguyên nhân lỗi positional-index trong thử nghiệm synthetic trước đó.

RGB: năm bước, và chỗ ImageNet normalization thực sự xảy ra



Shape sau loader

`[memory_size, 224, 224, 3] = [8, 224, 224, 3]`

Sau collate / unsqueeze(0):

`[B, T, H, W, C] = [1, 8, 224, 224, 3]`

Normalization KHÔNG nằm ở loader

ImageNet normalization xảy ra bên trong RGB backbone:

```

rgb_norm = (rgb - [0.485, 0.456, 0.406])
            / [0.229, 0.224, 0.225]

```

Chi tiết temporal-shape quan trọng

Loader cấp 8 RGB history frame nhưng chỉ 1 depth frame hiện tại. RGB-D encoder vì thế nhận 8×256 token RGB + 1×256 token depth — đúng bằng sức chứa positional $(\text{memory_size} + 1) \times 256$.

Depth: cùng một đơn vị mét, nhưng ba đường chuyển đổi khác nhau

Đường InternData-N1 (đã dùng)

`depth_m = raw_uint16 / 10000.0`

- Resize / pad về $224 \times 224 \times 1$
- Gán 0 cho $\text{depth} > 5.0 \text{ m}$ và $\text{depth} < 0.1 \text{ m}$
- Output float32, đơn vị mét

Cách đọc giá trị 0

Sau bước clip, giá trị 0 có thể là invalid, out-of-range hoặc padding. Không được hiểu là “có vật thể sát camera ở 0 mét”. Đây là lý do min/max depth phải in ra theo mét khi debug.

Ba nguồn depth trong repo — không được áp một phép scale cho tất cả

Nguồn	Depth đầu vào	Chuyển đổi trước khi vào S1
InternData-N1 PNG (offline loader)	uint16 encoded	Chia 10000.0 → mét; ngoài 0.1–5 m đặt về 0
Notebook / real-world async agent	Caller phải đưa sẵn theo mét	Chỉ resize; không nhân, không chia

Smoke test đã chạy đi theo dòng InternData PNG — không đi qua phép nhân 10 của Habitat. Câu “depth của S1 có đơn vị mét” chỉ đúng tại boundary của model.

Từ transform 4×4 sang hệ robot-local



Quy ước tọa độ

x	hướng tiến / lùi	<i>mét</i>
y	trái / phải — bên phải là âm	<i>mét</i>
theta	góc so với vector chuyển động ban đầu, atan2(cross, dot)	<i>radian</i>

point_goal là gì?

Là phần tử CUỐI CÙNG của chuỗi local xyt — tức vị trí và hướng tương đối của đích so với frame bắt đầu.

```
goal_point = [x, y, theta]
```

```
shape [1, 3] · float32
```

Toàn bộ phép biến đổi này được adapter tái sử dụng nguyên vẹn từ loader chính thức — không viết lại geometry, nhờ đó tensor đưa vào NavDP giống hệt lúc huấn luyện.

Từ cumulative pose sang diffusion target — và ngược lại

HUẤN LUYỆN

Loader tạo training target

- Chọn predict_size + 1 điểm trên chuỗi local
- Lấy sai phân giữa các điểm liên tiếp
- Nhân hệ số chuẩn hoá 4.0

```
delta[t] = pose[t+1] - pose[t]
training_action[t] = delta[t] * 4.0
```

SUY LUẬN

Model trả increment, code tích phân lại

- Diffusion sinh delta đã chuẩn hoá
- Chia lại cho 4 để về đơn vị mét
- Cumulative sum thành quỹ đạo

```
pose[t] = Σ ( delta_pred[0:t] / 4.0 )
```

Hệ số 4 đến từ đâu?

Nó là normalization tương ứng với bước tiến khoảng 0.25 m mỗi action. Hệ quả cần nhớ khi đọc log: output THÔ của diffusion là increment, còn output của API standalone đã được log lại là quỹ đạo CỘNG DỒN.

PHẦN 03 · TRỌNG TÂM

System 1 — NavDP standalone

Contract đã kiểm chứng · RGB-D backbone · diffusion · critic

I/O contract đã kiểm chứng bằng dữ liệu thật

ĐẦU VÀO

goal_point [1, 3] *float32*

Local (x, y, theta) — mét và radian

input_images [1, 8, 224, 224, 3] *float32, [0, 1]*

8 RGB history frame; frame thiếu được zero-pad

input_depths [1, 224, 224, 1] *float32, mét*

Chỉ depth của frame hiện tại

ĐẦU RA

positive trajectories

[K, 24, 3]

Critic giảm dần — ứng viên tốt nhất

negative trajectories

[K, 24, 3]

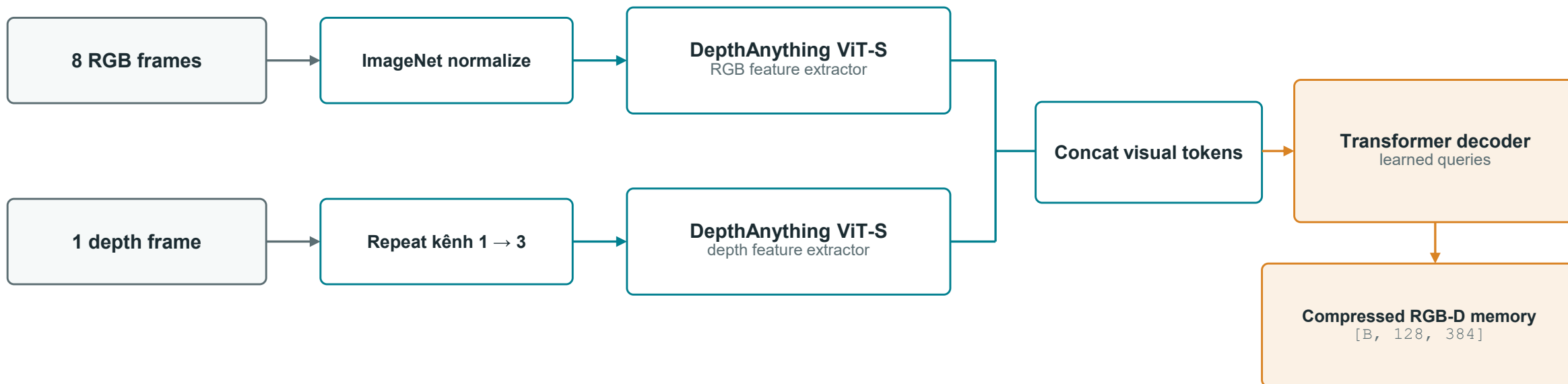
Critic tăng dần — ứng viên kém nhất

$K \leq 8$, đồng thời phụ thuộc $\text{sample_num} \times B$

NavDPNet là episode-stateless

Mỗi lần gọi `predict_pointgoal_batch_action_vel` đều nhận đầy đủ goal và observation tensor. Diffusion scheduler là state của model, không phải lịch sử điều hướng. Toàn bộ state điều hướng nằm ở tầng agent, không nằm trong NavDPNet.

RGB-D backbone: từ ảnh sang token nhớ



Phép đếm token

Mỗi frame 224×224

→ 256 patch token

8 RGB + 1 depth

→ 9×256 token, khớp sức chứa $(\text{memory_size} + 1) \times 256$

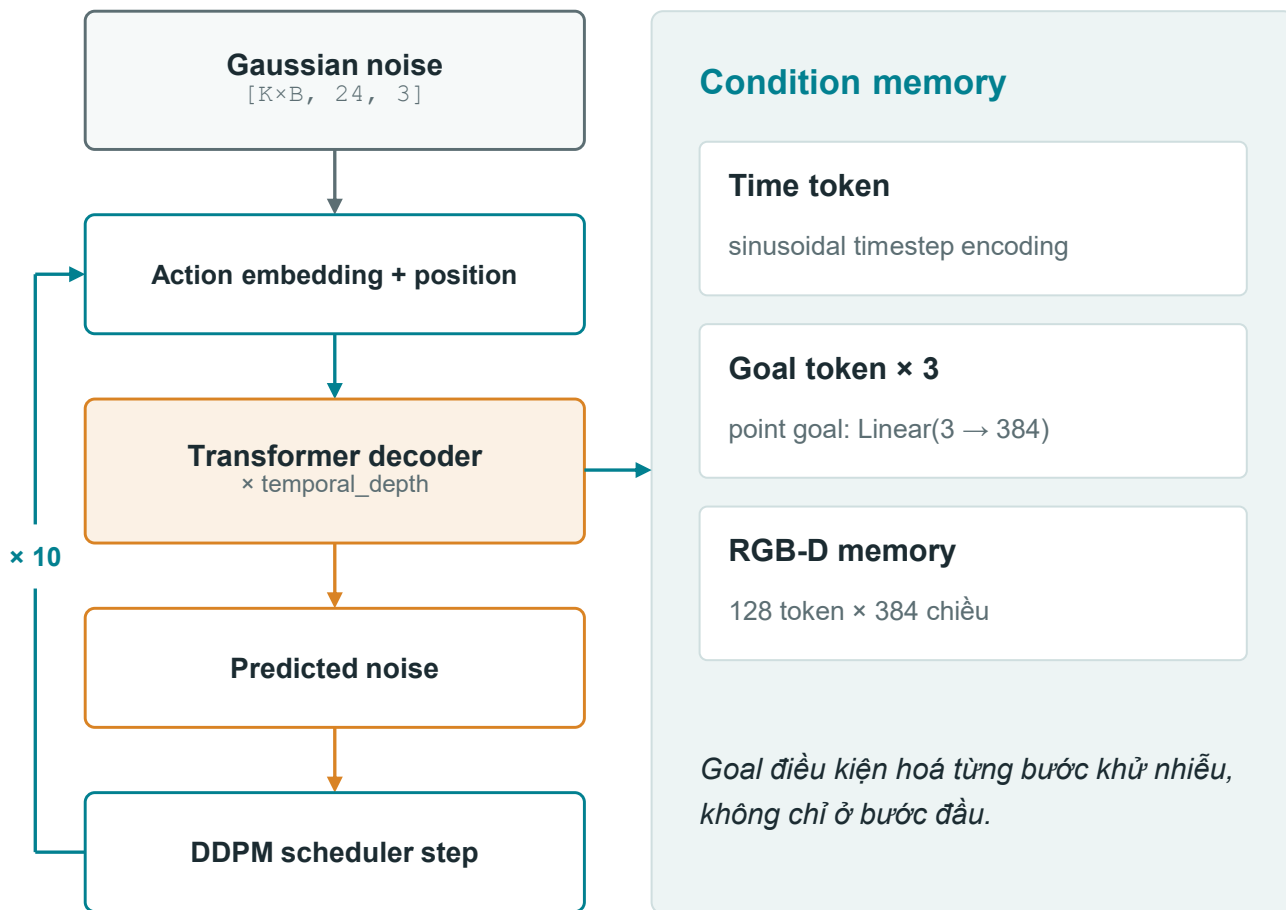
Learned queries nén

→ $\text{memory_size} \times 16 = 128$ token, mỗi token 384 chiều

Đừng nhầm

Depth 1 kênh được lặp thành 3 kênh để đi qua ViT. Đó là channel conversion nội bộ — KHÔNG phải lặp depth thành 8 frame theo thời gian.

Sinh quỹ đạo bằng diffusion



Vì sao sinh nhiều ứng viên?

Từ một quan sát có thể tồn tại nhiều đường đi đều hợp lý — đi vòng trái hay vòng phải quanh một chiếc ghế đều đúng. Diffusion giữ được tính đa phương án thay vì ép model trả về một quỹ đạo trung bình vô nghĩa.

24

bước dự đoán

10

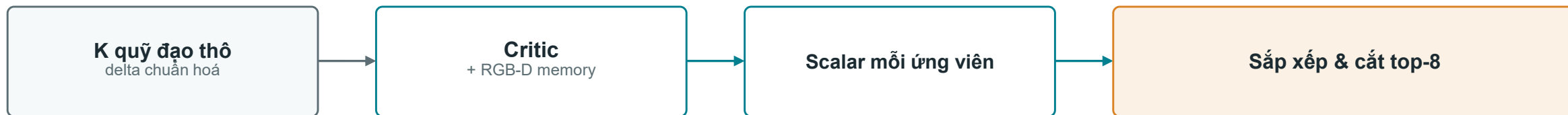
DDPM timestep

3

kênh (x, y, yaw)

 $\text{naction} \sim N(0, I), \text{ shape } [K \times B, 24, 3]$

Critic xếp hạng và định dạng output standalone



positive_trajectory

Sắp theo critic giảm dần, lấy tối đa 8 ứng viên điểm cao nhất.

`shape [min(K, 8), 24, 3]`

negative_trajectory

Sắp theo critic tăng dần, lấy tối đa 8 ứng viên điểm thấp nhất.

`shape [min(K, 8), 24, 3]`

Bước cuối trước khi trả về

```
trajectory = cumsum(naction / 4.0, dim=1)
```

Vì vậy output của API standalone là quỹ đạo cộng dồn, đơn vị mét/radian.

Bẫy khi K nhỏ

Với `sample_num = 2`, cả positive và negative đều chứa CÙNG hai ứng viên, chỉ khác thứ tự. Khi đó min/max của hai nhóm trùng nhau — đây là hành vi đúng, không phải lỗi.

PHẦN 04

Điểm ghép

$S2 \rightarrow \text{latent} \rightarrow S1$

Learned latent queries · token compressor · dual async

Input, prompt và hai nhánh output

S2Input giữ gì?

- instruction — câu lệnh điều hướng
- rgb — frame hiện tại
- depth — không qua processor Qwen, nhưng lưu làm memory cho S1
- pose, look_down, idx, should_infer

Prompt trong code

```
You are an autonomous navigation
assistant.
Your task is to <instruction>.
Where should you go next to stay
on track?
Please output the next waypoint's
coordinates in the image.
Please output STOP when you have
successfully completed the task.
```

Chọn ảnh lịch sử

- Append frame hiện tại vào rgb_list
- Chọn $\leq \text{num_history}$ index phân bố đều bằng linspace + unique
- Thêm frame hiện tại ở cuối
- Chèn <image> placeholder tương ứng

Là subsampling dài hạn, không phải 8 frame liền nhau.

Parser: nếu text có chữ số → pixel waypoint; nếu không → token hành động (STOP / forward / left / right / look-down)

Nhánh A — discrete action

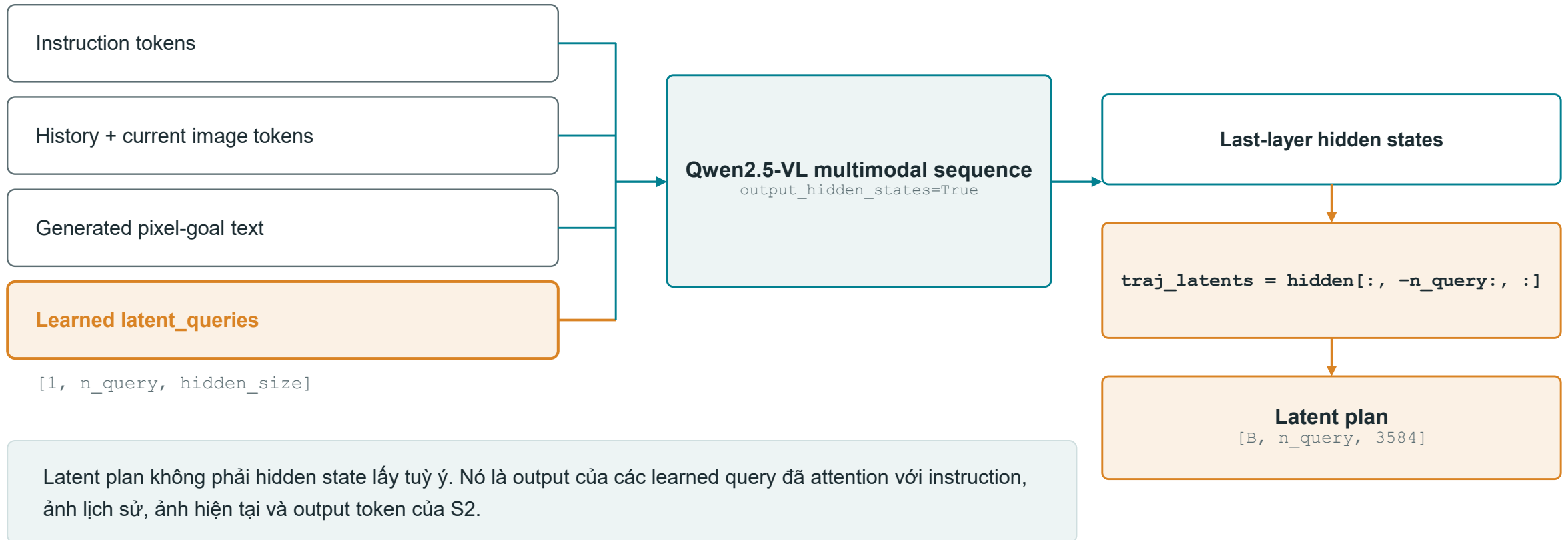
System 1 bị bypass hoàn toàn. Agent phát thẳng action ra environment.

Nhánh B — pixel goal + latent plan

```
pixel_goal = [coord[1], coord[0]] = [y, x]
```

Model tiếp tục sinh latent plan cho System 1.

Learned latent queries: latent plan được tạo ra thế nào

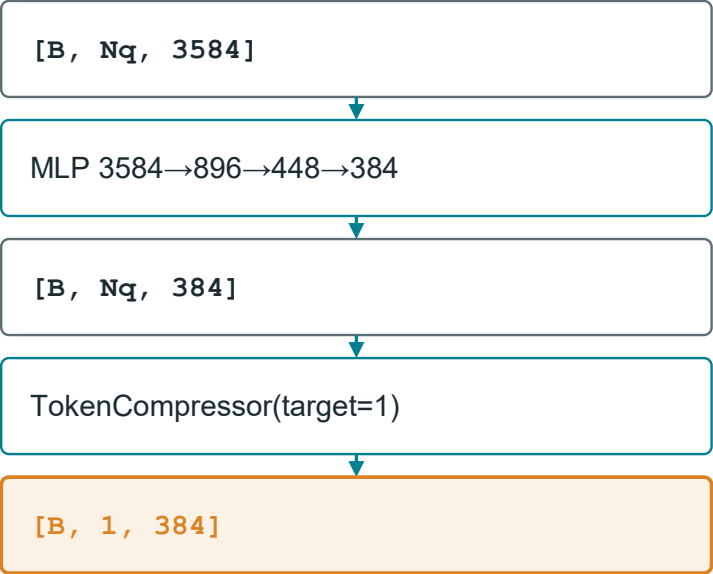


Vì sao load bằng Transformers thuần có thể sai?

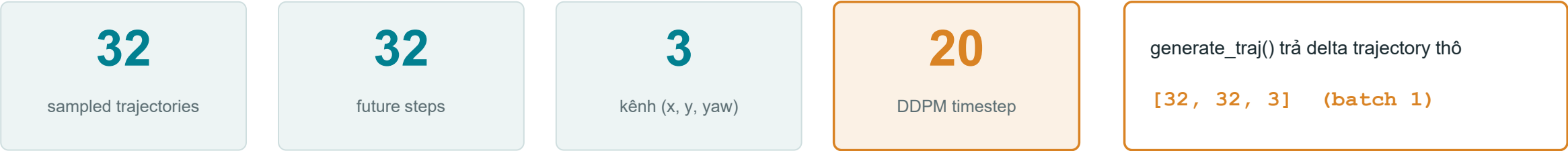
Checkpoint chứa latent_queries và toàn bộ module navdp.* — phải load bằng custom InternVLN1ForCausalLM. Ép vào AutoModel Qwen thuần, các weight này bị báo unexpected/ignored: VLM vẫn chạy nhưng đường nối latent plan → NavDP đã mất.

Latent plan đi vào NavDP dual-system

Nén latent thành goal token



Output diffusion dual



RGB-D dual async: memory_size = 2



34 condition token

$2 \text{ RGB} \times 256 + 2 \text{ depth} \times 256$
= 1024 visual token

Learned queries nén còn
memory_size $\times$ 16 = 32 token

Condition = 1 time + 1 goal + 32
= 34 token, mỗi token 384 chiều

Từ quỹ đạo sang hành động của robot

Continuous — notebook real-world

- Chia dx, dy cho 4 để bỏ chuẩn hoá
- Cumulative sum thành 32 candidate path
- Lấy trung bình theo chiều sample
- Trả một quỹ đạo liên tục $[T+1, 2]$ bắt đầu tại $[0, 0]$

Discrete — evaluator / agent

- Cũng tạo mean trajectory, rồi mô phỏng follower đơn giản
- Mỗi bước tiến 0.25 m · mỗi lần quay 15°
- Lookahead mặc định 4 trajectory point
- Dừng khi cách goal ≤ 0.2 m hoặc bước tiếp làm xa goal hơn

Action vocabulary

0 STOP

1 Move forward 0.25 m

2 Turn left 15°

3 Turn right 15°

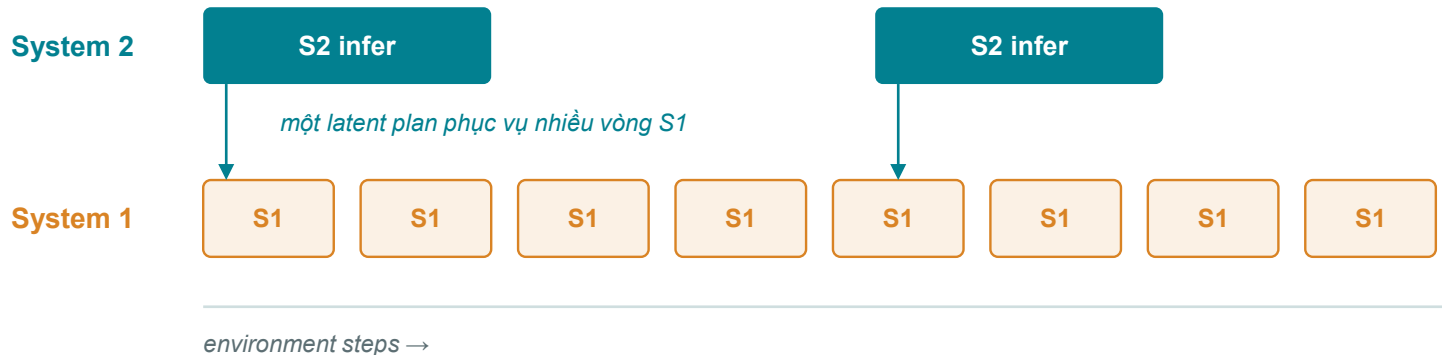
5 Look down 30°

-1 No-op / wait sentinel

Cache action

`s1_step_latent()` loại action 0 và chỉ giữ tối đa 4 action đầu. Agent trả action đầu ngay, cache phần còn lại vào `s2_output.output_action` để phát ở các environment step kế tiếp.

S2 chậm, S1 nhanh — và vì sao reset() là bắt buộc



Evaluator agent

S2 chạy lại khi:

- hết action/pixel/latent hợp lệ
- $\text{dual_forward_step} \geq \text{sys2_max_forward_step}$ (mặc định 8)
- cần look-down

Notebook real-world

Đơn giản hoá bằng $\text{plan_step_gap} = 4$: quá 4 frame kể từ lần S2 gần nhất (hoặc chưa có output) thì chạy S2; giữa các lần đó chỉ append frame qua $\text{step_no_infer}()$ rồi chạy S1 với latent đang giữ.

reset() xoá gì?

- `rgb_list`, `depth_list`, `pose_list`
- `conversation_history`, `llm_output`
- `pixel goal`, `latent plan`, `cached action`
- S1/S2 input-output dataclass
- `episode & forward counters`, `look-down state`

Nếu quên reset

- S2 dùng ảnh của episode trước
- Latent cũ điều khiển S1 trong scene mới
- `Cached action` cũ phát trước khi model kịp infer
- Chỉ số ảnh và `scheduling` lệch nhau

PHẦN 05

Bằng chứng, lỗi thường gặp và tiến độ

Cái gì đã chứng minh được, cái gì vẫn là giả định

Kết quả chạy thật trên RTX 4050 Laptop GPU

1.59 s

model-call latency

575.5 MiB

peak CUDA allocation

0

missing checkpoint key

122

RGB / depth frame trong episode

Input đã kiểm tra

Tensor	Kết quả quan sát
goal_point	[1, 3] float32, finite · [-0.396, 1.877]
RGB history	[1, 8, 224, 224, 3] float32 · [0, 0.827]
current depth	[1, 224, 224, 1] float32 · [0, 4.998] m

Output đã kiểm tra

Tensor / metric	Kết quả quan sát
positive traj.	[2, 24, 3] float32, finite
negative traj.	[2, 24, 3] float32, finite
extra ckpt keys	18 · đều thuộc template decoder_layer.*

Độc số liệu cho đúng

- Range output trộn mét (x, y) và radian (theta) → chỉ dùng làm finite/sanity check, không phải một metric vật lý duy nhất.
- Latency chỉ đo inference sau khi model và dataset đã load; PyTorch allocated memory không bằng tổng VRAM trong nvidia-smi.
- Cảnh báo “xFormers not available” là warning hiệu năng — không chặn dựng model hay inference.

Dual-system 7B chạy end-to-end open-loop trên Kaggle T4×2 (23/07)

16.78 GB

checkpoint · VRAM 7.57+9.21 GB

98.8 M

tham số NavDP (S1) đã nạp

0

tham số meta còn sót sau load

101/128

frame S2 ra pixel + latent

GATE 1 — load & verify

Phép kiểm	Kết quả quan sát
system1	navdp_async (config có sẵn, không patch)
latent_queries	[1, 4, 3584] · n_query = 4
tokenizer	slow→fast OK (transformers 4.51.0)

GATE 2 + 3 — S2 → latent → S1

Tensor / metric	Kết quả quan sát
S2 latent	[B, 4, 3584] finite (nhánh pixel-goal)
s1_traj	(33, 2) không NaN · predict_size = 32
s1_actions	∈ {1,2,3}, ≤4 · trên 101 frame latent

Độc số liệu cho đúng

- Open-loop = feed observation tĩnh vln_ce vào policy (pose = ma trận đơn vị); KHÔNG phải benchmark SR/SPL closed-loop.
- device_map='auto' + attn 'sdpa' là 2 override bắt buộc so với code gốc (pin 1 GPU + flash_attention_2) để chạy trên T4×2.
- GATE 1/2/3 đều PASS — lần đầu xác nhận runtime cơ chế config.system1 dựng NavDP (bản -w-NavDP có sẵn field này).

System 2 open-loop: chỉ số thật và cơ chế look-down 2 nhịp

41.8 px

pixel-goal L2 trung bình (median 20)

37 %

action accuracy (n=27)

79 %

frame ra pixel+latent (101/128)

35.5 s

thời gian S2 mỗi frame (2 nhịp)

Cơ chế look-down 2 nhịp

Nhịp	Điều gì xảy ra
Nhịp 1	model trả '↓' = action 5 (xin cúi camera)
Nhịp 2	gọi lại s2_step(look_down=True) → ra pixel+latent
Kết quả	101/128 frame ra pixel qua đúng cú 2 nhịp

Toạ độ & nhánh output

Điểm	Kết quả thật
Không gian toạ độ	model nhả 'u v' trong 640×480 gốc
KHÔNG scale 384→640	scale nhằm thổi L2 41.8 → 206 px
pixel lưu [row,col]	= [v, u]; đảo lại khi so với GT goal

Độc số liệu cho đúng

- L2 41.8 px trên 640×480: 12 outlier >100px (3 là frame 0–2 ep0 chưa có history) — median 20 px sát thực tế hơn.
- Action acc 37% trông thấp nhưng 10/17 lỗi là 'STOP sớm' dồn cuối episode; chỉ 7 lỗi điều hướng thật giữa episode.
- S2 ~35.5 s/frame gồm ~2 lần generate/frame (look-down) — cả run 128 frame ≈ 75 phút trên T4×2.

Notebook chính thức chứng minh gì — và không chứng minh gì



Notebook CHỨNG MINH

- Cách load custom InternVLAN1ForCausalLM
- API InternVLAN1AsyncAgent.step(rgb, depth, pose, instruction, intrinsic)
- History frame, plan_step_gap, pixel output, latent, quỹ đạo liên tục
- Depth phải aligned với RGB và có đơn vị mét



Notebook KHÔNG chứng minh

- Chất lượng S1 trên sample đi kèm — sample real-world quên ghi depth
- Cell demo dùng depth giả toàn giá trị 10 m
- Chính notebook cảnh báo quỹ đạo sẽ ngắn và yêu cầu dữ liệu tự thu phải ghi cả aligned RGB/depth

Kết quả standalone bổ sung đúng phần còn thiếu

Lần chạy real-data dùng depth thật từ InternData-N1, đúng loader và đúng đơn vị mét — tức là lắp vào chỗ notebook để trống. Nhưng nó vẫn KHÔNG thay thế được một lần chạy dual-system 7B end-to-end.

Các failure mode cần nhớ

Hiện tượng	Nguyên nhân thường gặp	Cách kiểm tra
S1 output rất ngắn / vô lý	Depth giả, sai đơn vị hoặc invalid	In min/max depth theo mét; kiểm tra aligned RGB-D
Lỗi shape positional embedding	Nhầm contract standalone và dual async; lặp depth sai	Standalone: 8 RGB + 1 depth · dual async: 2 RGB + 2 depth
VLM chạy nhưng không có latent/NavDP	Load bằng Qwen/AutoModel thuần	Dùng custom InternVLAN1ForCausalLM; kiểm tra custom keys
Pixel bị vẽ lệch	Nhầm [x, y] với [y, x]	Parser lưu [y, x]; OpenCV circle nhận (x, y)
Episode mới hành động sai ngay đầu	Quên reset()	Xoá history, latent, cached action và counters
Preview loader lỗi image_index	Metadata mới lồng dưới stats	Dùng compatibility discovery như adapter Ngày 2
Obstacle critic không thật	Preview thiếu meta/pointcloud.ply	Không đánh đồng path.ply với obstacle point cloud
positive/negative có cùng range	sample_num quá nhỏ so với top-8	Với K = 2, cả hai nhóm chứa cùng hai candidate

Nhánh A (standalone) và nhánh B (dual-system open-loop) đã đóng

NGÀY 1 — MÔI TRƯỜNG

- WSL2 nhận GPU, `torch.cuda.is_available() == True`
- Env internnav-s1 hoạt động, import internnav thành công
- Khởi tạo submodule LongCLIP và diffusion-policy
- Có checkpoint DepthAnything và NavDP standalone
- Smoke inference GPU local đã chạy
- Bổ sung dependency loader InternData, pip check sạch

NGÀY 2 — NHÁNH A (NAVDP)

- Tải chọn lọc InternData-N1: sample vln_n1 (tar.gz) + vln_ce/vln_pe — preview/ không tồn tại
- Hiểu và ghi lại cấu trúc LeRobot/InternNav
- Tìm đúng sample vln_n1 có đủ RGB, depth, camera matrices
- Dùng preprocessing/geometry chính thức thay dummy tensor
- Chạy NavDP standalone trên RGB/depth/goal thật
- Xác minh shape, dtype, range, finite, latency, CUDA memory
- Có script tái sử dụng, JSON log và tài liệu I/O

CÒN LẠI — CLOSED-LOOP / EVAL

- Chạy closed-loop trong Habitat/Isaac → chưa có SR/SPL thật
- Xác nhận trên split chính thức R2R val_unseen / MP3D
- Dựng env eval đầy đủ + đo latency S2/S1 trên server $\geq 24\text{GB}$

Đây là luồng cloud/eval của nhóm, không phải lỗi của pipeline local.

Nhánh B (dual-system 7B) đã chạy end-to-end open-loop ngày 23/07 — sớm hơn dự kiến. Việc còn lại thuần túy là closed-loop trên simulator, không phải lỗi pipeline.

Chín điểm đã xác minh trên checkpoint 7B (23/07)

1 model_type = internvla_n1

2 hidden_size = 3584, num_hidden_layers = 28

3 system1 = navdp_async

4 n_query = 4 (đo thật trên -w-NavDP)

5 Tồn tại key kết thúc bằng latent_queries

6 Có đủ nhiều key navdp.* và biết prefix chính xác

7 Custom model load các key này không bị bỏ qua

8 Một inference S2 sinh pixel + latent finite

9 Một inference S1 nhận latent + RGB-D và sinh trajectory finite

Đây là gate của nhánh cloud/dual-system.

✓ Toàn bộ 9 điểm PASS ngày 23/07 (PL-E2/E3). Dual-system open-loop hoàn tất; còn lại chỉ là closed-loop SR/SPL.

Source map trong repo

Nội dung	File chính
Notebook inference chính thức	<code>InternNav-TTS/scripts/notebooks/inference_only_demo.ipynb</code>
Real-world S2 → latent → S1	<code>internnav/agent/internvla_n1_agent_realworld.py</code>
Evaluator async state machine	<code>internnav/agent/internvla_n1_agent.py</code>
Prompt, parser, s1_step_latent	<code>internnav/model/basemodel/internvla_n1/internvla_n1_policy.py</code>
Learned queries, generate_latents, generate_traj	<code>internnav/model/basemodel/internvla_n1/internvla_n1.py</code>
Build NavDP bên trong custom model	<code>internnav/model/basemodel/internvla_n1/internvla_n1_arch.py</code>
Dual NavDP latent-conditioned diffusion	<code>internnav/model/basemodel/internvla_n1/navdp.py</code>
Standalone NavDP + critic	<code>internnav/model/basemodel/navdp/navdp_policy.py</code>
RGB-D tokenization	<code>internnav/model/encoder/navdp_backbone.py</code>
LeRobot preprocessing / geometry	<code>internnav/dataset/navdp_lerobot_dataset.py</code>
Action conversion + dataclasses	<code>internnav/model/utils/vln_utils.py</code>
Adapter Ngày 2 · report real-data	<code>MH_week2/scripts/run_navdp_real_sample.py</code> · <code>logs/navdp_real_sample.json</code>

MỘT CÂU ĐỀ THUYẾT TRÌNH

System 2 dùng instruction và lịch sử ảnh để tạo waypoint cùng learned latent plan; System 1 nén latent đó thành goal token, kết hợp RGB-D lúc lập plan và RGB-D hiện tại, rồi dùng diffusion sinh nhiều local trajectory.

Trajectory được bỏ chuẩn hoá, tích phân và chuyển thành đường liên tục hoặc action rời rạc để robot thực thi. Nhờ partial async, một quyết định chậm của S2 có thể phục vụ nhiều vòng điều khiển nhanh của S1.

● Đã chứng minh

NavDP standalone + dual-system open-loop (S2→latent→S1) đều chạy thật

● Đã đọc kỹ code

Toàn bộ đường S2 → latent → S1 và post-processing

● Bước kế tiếp

Chạy closed-loop trong Habitat/Isaac để có SR/SPL (open-loop đã xong 23/07)